Paper 2 - Correction Surface

CQE-paper-02
CQECMPLX Formal Paper Series
Review-facing PDF built 2026-06-12

Construction Basis

Treat failure, mismatch, and nonlinear residue as positive correction data rather than dismissal.

This PDF is the clean paper artifact. Source extracts, kernels, receipts, tool notes, workbook material, and package bindings are used as construction evidence; they are not treated as separate review-facing papers.

Evidence Inputs

- top-level review source: Papers\Source\CQE-paper-02.md
- paper kernel manifest: production\paper-kernels\papers\CQE-paper-02\paper_kernel_manifest.json
- body: production\papers\CQE-paper-02\PAPER-BODY.md
- source: production\papers\CQE-paper-02\SOURCE.md
- formal: production\papers\CQE-paper-02\01-CQE-formal\FORMAL.md
- tool: production\papers\CQE-paper-02\02-CQE-tool\TOOL.md
- tool_runner: production\papers\CQE-paper-02\02-CQE-tool\run.py
- workbook: production\papers\CQE-paper-02\03-CQE-workbook\WORKBOOK.md
- recovered_output: production\lib-forge\recovered\papers_output\CQE-paper-02.md

Paper 2 - Correction Surface

Abstract

Paper 2 proves the first correction theorem in the CQECMPLX sequence. Paper 1 established the minimal local carrier `(L,C,R)`. Paper 2 asks what happens when a simple boundary-only transport is not enough to reproduce the requested local update.

For the binary Rule 30 / Rule 90 case, the answer is exact. The Rule 30 update decomposes into the Rule 90 boundary update plus one center-bearing residue:

```
Rule30(L,C,R) = Rule90(L,R) xor corr(L,C,R)
corr(L,C,R)   = C and not R
```

The correction fires on exactly two of the eight binary LCR states:

```
(0,1,0)
(1,1,0)
```

This paper proves the decomposition and records the scientific meaning of the residue. A correction is not a license to promote a failed route into a proof. It is positive data only when it is typed, localized, replayable, and routed to the next claim that can consume it.

Claims

Claim 2.1. On `{0,1}^3`, Rule 30 factors through Rule 90 plus the exact correction `C and not R`.

Claim 2.2. The correction surface is exactly the two-state set `{(0,1,0), (1,1,0)}`.

Claim 2.3. Under the D4 axis/sheet chart used by the later triality paper, the correction firing states project to `(2,0)` and `(3,1)`.

Claim 2.4. A nonzero correction is an obligation-bearing residue, not a closed proof of the route that failed.

Definitions

Let the local state be the binary LCR carrier:

```
s = (L,C,R) in {0,1}^3
```

Define the Rule 30 local update:

```
R30(L,C,R) = L xor (C or R)
```

Define the Rule 90 boundary update:

```
R90(L,R) = L xor R
```

Define the correction map:

```
corr(L,C,R) = C and not R
```

The **correction surface** is the set of states for which `corr = 1`, together with the receipt row that records the source state, the residue value, the failed route, and the next legal intake.

Theorem 2.1 - Correction Decomposition

For every `(L,C,R) in {0,1}^3`,

```
R30(L,C,R) = R90(L,R) xor corr(L,C,R)
```

and `corr(L,C,R) = 1` if and only if `(L,C,R)` is `(0,1,0)` or `(1,1,0)`.

Proof

Over Boolean values,

```
C or R = C xor R xor (C and R)
```

Therefore:

```
R30(L,C,R)
  = L xor (C or R)
  = L xor C xor R xor (C and R)
  = (L xor R) xor (C xor (C and R))
  = R90(L,R) xor (C and not R)
```

The term `C and not R` equals `1` exactly when `C = 1` and `R = 0`. The value of `L` is free in that condition, so the firing states are exactly:

```
(0,1,0)
(1,1,0)
```

This proves the decomposition and the correction surface.

D4 Projection

The production verifier records the D4 axis/sheet labels for all eight LCR states. On the two correction states, the projection is:

```
(0,1,0) -> (2,0)
(1,1,0) -> (3,1)
```

Paper 2 does not prove full D4 triality. It exports only the finite projection needed by Paper 3. The stronger registration claim belongs to Paper 3.

Receipt

The primary executable receipt is:

```
production/formal-papers/CQE-paper-02/verify_correction_surface.py
production/formal-papers/CQE-paper-02/correction_surface_receipt.json
```

The receipt status is `pass`. It verifies:

```
state_count_is_8                        = true
identity_holds_for_all_states           = true
firing_states_exact                     = true
d4_projection_exact                     = true
nonzero_residue_is_obligation_not_proof = true
```

Falsifiers

The paper fails if any of the following occur:

```
Rule30 != Rule90 xor corr for any binary LCR state
corr fires outside {(0,1,0),(1,1,0)}
corr fails to fire on either (0,1,0) or (1,1,0)
the D4 projection of the firing rows is changed without a new receipt
```

```
a nonzero residue is counted as a closed proof
```

Prediction

Any independent implementation that enumerates `{0,1}^3` must find six non-firing states and two firing states. It must also record the two firing states as obligations or next-route inputs, not as proof closures.

Role in the Suite

Paper 1 provides the carrier. Paper 2 proves that failure against a simpler boundary update has a precise finite residue. Paper 3 receives the two correction rows as registered coordinates, and Paper 4 turns failed joins into typed boundary repair constraints.

The important transfer is:

```
carrier state -> correction residue -> registered coordinate -> repair input
```

Open Obligations

- Expose the verifier through the installable kernel/API interface.
- Keep the Paper 3 D4 chart synchronized with this paper's two exported
correction coordinates.
- Extend the receipt schema so later papers can import correction rows
directly rather than re-parsing the Paper 2 receipt.

Conclusion

Paper 2 proves that the first nontrivial correction surface is not vague. It is the Boolean term `C and not R`, it fires on exactly two states, and it exports those rows as typed residue. The correction layer lets the suite keep failed structure visible without pretending the failure has already become a closed theorem.